fid-synth___fidelity-flat-f-k4

An AgentMPI run analysis

Abstract

This is the one-round, fifteen-deep cell of the surrogate series, and it is the clearest available demonstration that `flat` is not a one-shot p-way fold. The collective reports `rounds: 1` and `fold_depth: 15`, and the trace shows why both are true at once: fifteen ranks send their leaf reports to rank 0 in the first 43 ms, and then rank 0 issues fifteen `agent.call` events labelled `merge:d1` through `merge:d15`, one binary application per contribution. Communication is a single round; the fold is as deep as the chain's. The executor is `function` × 16, so this measures the protocol and the surrogate kernel, not agent behaviour — and here the kernel misbehaves spectacularly. Because it copies both inputs verbatim into its output, and because at the root every application's left input is the previous accumulator, the output size doubles fifteen times: 177, 370, 644, 1,059, 1,756, 3,017, 5,406, 10,052, 19,192, 37,242, 73,450, 145,733, 290,166, 578,900 and finally 1,156,234 tokens, for 2,323,398 output tokens and a modelled $38.36 — 545 times the k-ary cell's $0.0704 for the same collective over the same data. This cell also scores the campaign's highest retention, 0.875, while having its greatest depth, which is worth stating plainly and then discounting: the reason is that `flat`'s fifteen prompts all have different lengths, and the surrogate reseeds from `seed + len(prompt)`, so its draws are effectively independent where the binomial tree's five identical depth-1 prompts all failed together. Retention here is a property of prompt lengths, not of fold shape.

1 What this run is

property	value
run	`fid-synth__fidelity-flat-f-k4`
experiment	–
campaign label	–
declared world size	16
ranks appearing in the log	16
executors	`function` × 16
events	95
wall time	0.73 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.77×	total busy time over wall time
parallel efficiency	4.8%	achieved parallelism per rank
idle fraction	95.2%	rank-seconds paid for and unused
peak ranks busy	1	of 16 in the world
serial fraction of active time	100.0%	time with exactly one rank busy
load imbalance	16.00×	slowest rank's busy time over the mean
coordination share	7.1%	rank-seconds blocked in collectives, of those available
coordination in flight	98.8%	wall clock with any rank inside a collective
rank reattachments	0	fresh agent processes that took over a rank role
agent calls	15	invocations that reached a model
agent latency p50 / p95	0.01 / 0.21 s	per invocation
messages	15	15 eager, 0 rendezvous
tokens sent	2,133	0 deferred under rendezvous
tokens in / out	1,171,156 / 2,323,398	prompt and completion
cost	$38.3644	0.0165 $/kTok out

3 What the analysis notices

- [note] Occupancy is not measurable for this run: its executors (function) emit no broker claim events, so busy time, achieved parallelism, and the idle fraction are artefacts of the instrumentation rather than properties of the run.
- [note] Load imbalance is 16.00×: the slowest rank was busy far longer than the mean.
- [ok] All 1 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.
- [ok] All 1 checkable collective(s) also matched the predicted round count.

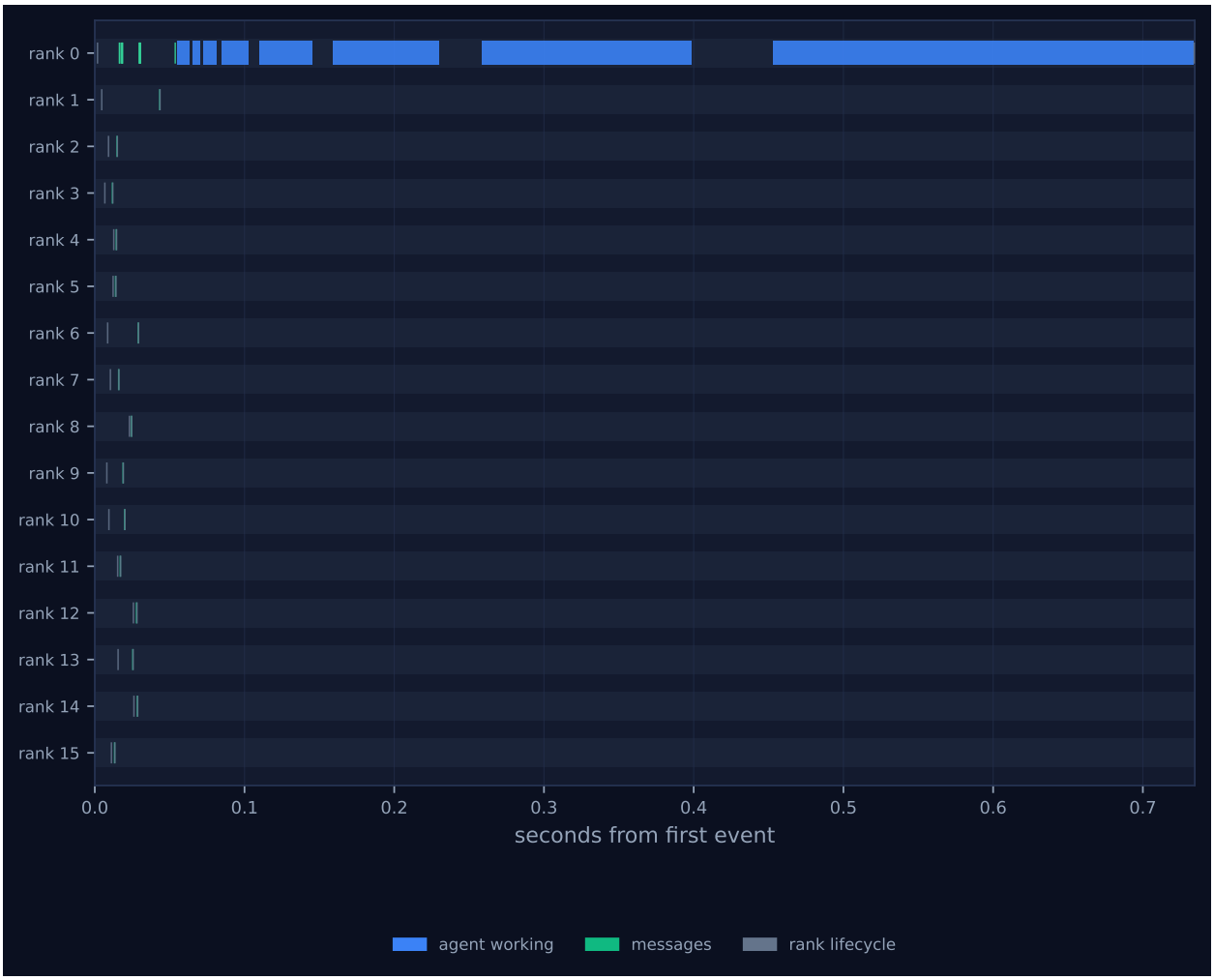

Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

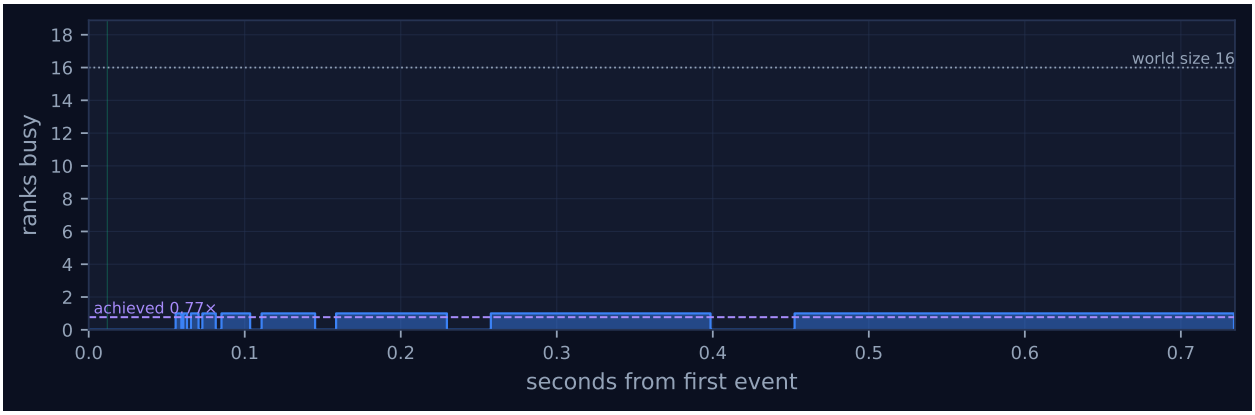

Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

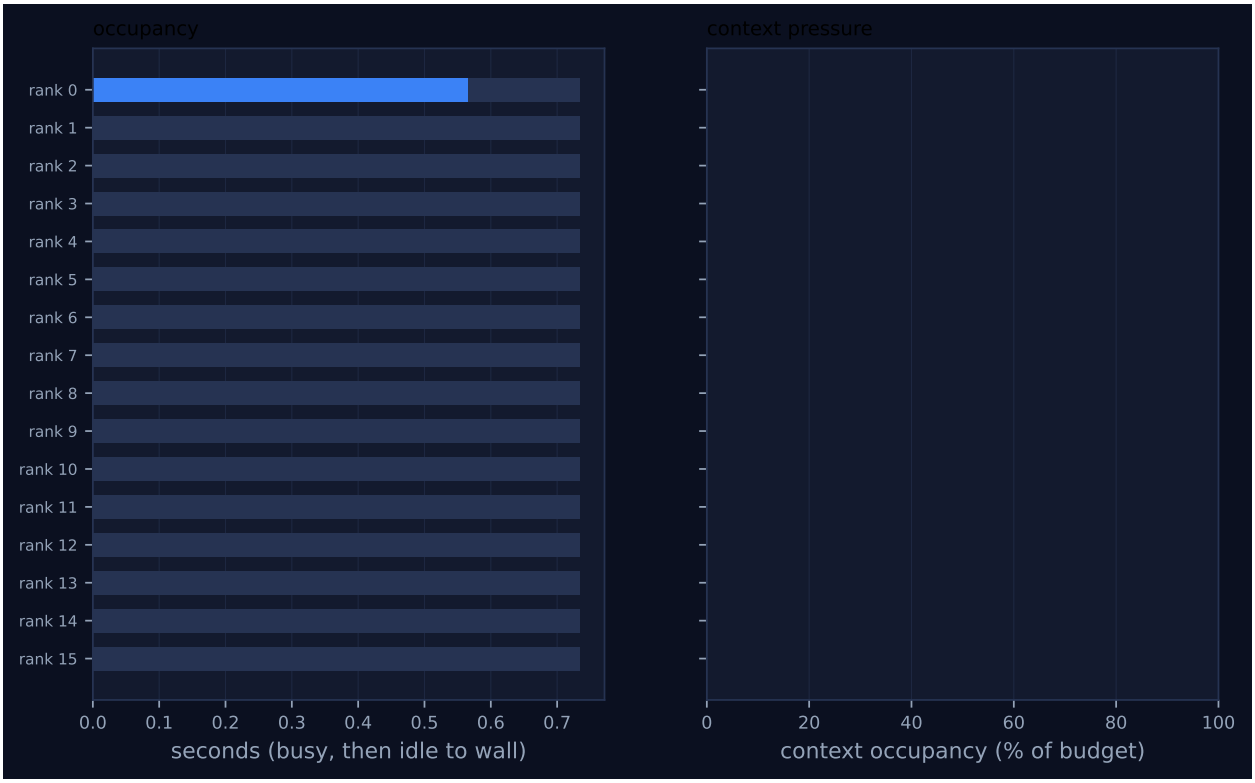

Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

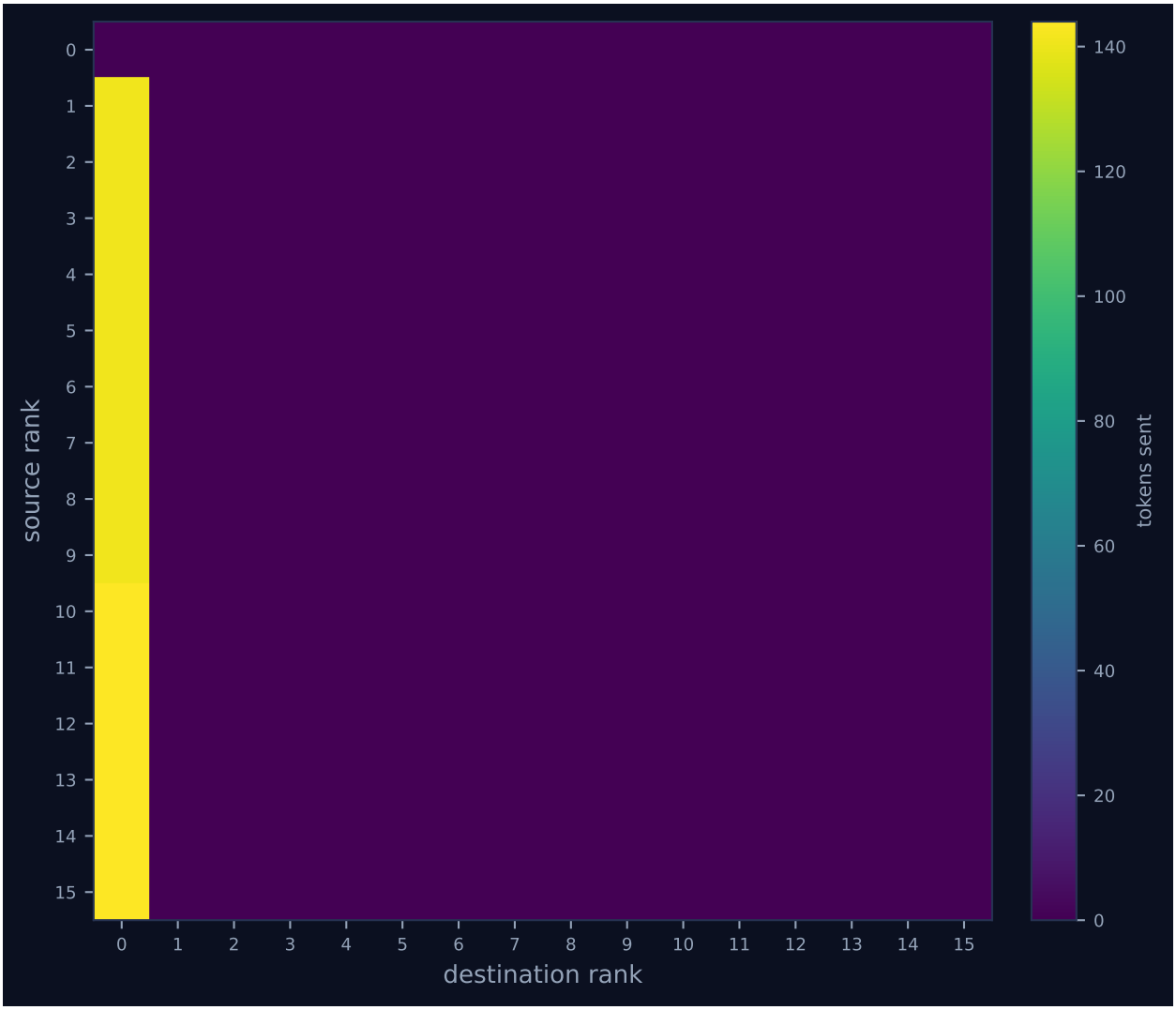

Figure 4: Communication matrix: tokens sent from each rank to each rank. The algorithm's shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

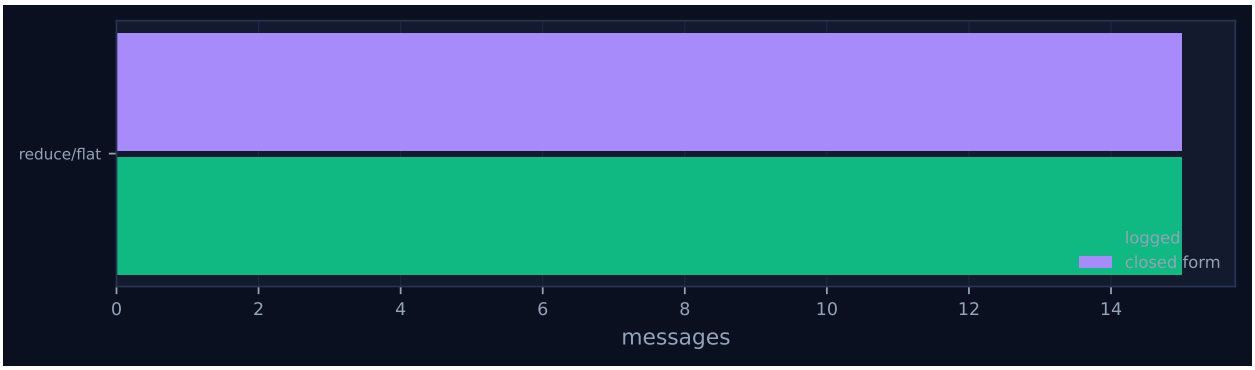

Figure 5: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	0.57	77.3	11	15	0	15	0	0.0	finalized
1	0.00	0.0	0	0	1	0	141	0.0	finalized
2	0.00	0.0	0	0	1	0	141	0.0	finalized
3	0.00	0.0	0	0	1	0	141	0.0	finalized
4	0.00	0.0	0	0	1	0	141	0.0	finalized
5	0.00	0.0	0	0	1	0	141	0.0	finalized
6	0.00	0.0	0	0	1	0	141	0.0	finalized
7	0.00	0.0	0	0	1	0	141	0.0	finalized
8	0.00	0.0	0	0	1	0	141	0.0	finalized
9	0.00	0.0	0	0	1	0	141	0.0	finalized
10	0.00	0.0	0	0	1	0	144	0.0	finalized
11	0.00	0.0	0	0	1	0	144	0.0	finalized
12	0.00	0.0	0	0	1	0	144	0.0	finalized
13	0.00	0.0	0	0	1	0	144	0.0	finalized
14	0.00	0.0	0	0	1	0	144	0.0	finalized
15	0.00	0.0	0	0	1	0	144	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	model
reduce	flat	–	16	16	1	1	15	15	2,133	0.73	0.72	✓

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

Figure 1 has one occupied lane. Ranks 1 to 15 appear as two ticks each inside the first 43 ms — a `msg.send` and a `coll.reduce` recording `fold_depth: 0` — and then nothing; rank 0 carries a train of fifteen marks from $t = 0.054$ s to $t = 0.734$ s whose widths grow visibly toward the right, the last one occupying 0.281 s of the 0.735 s run on its own. That is the flat reduction drawn literally: gather everything, then fold it all at the root. The 16.00× imbalance is the worst attainable at $p = 16$ and is correctly reported; the flagged occupancy figures should be ignored, since a `function` executor emits no broker claim events and the generated findings say so.

`coordination_share` of 7.1% is the lowest in my set and it is misleading in a way worth naming, because this is the most wasteful arrangement I have of a rank pool. The fifteen non-root ranks

return from the reduce in milliseconds, so their idleness never enters `collective_rank_seconds`; the chain sibling, whose ranks block on receives for the whole run, reports 23.8% for a strictly better-balanced pattern. A reader optimising on that metric would prefer this cell, and would be wrong.

The communication matrix, Figure 4, is a single column of fifteen edges at 141 or 144 tokens each, 2,133 in total — the least traffic of any cell at this p, against the binomial tree's 5,416 and the chain's 125,073. That is the trade the cost model predicts and the run confirms: one round of communication bought by putting all $p - 1$ applications on one rank's critical path. What the matrix cannot show, and what makes this cell pathological, is that the accumulator never crosses the fabric and therefore never appears in the traffic figures at all — so the 1,156,234-token object rank 0 produced is invisible to every message-based metric in the run.

7 Interpretation

The important structural point is the one the campaign has previously got wrong. A flat reduce is naturally described as folding all p inputs at the root in a single step, which would make it the shallowest arrangement available and the natural upper bound on retention. The implementation does no such thing: the `flat` branch of `reduce` gathers all p contributions, sorts them by source rank because the operator is declared non-commutative, and then applies the *binary* kernel $p - 1$ times in sequence, setting `fold_depth = p - 1 = 15` while leaving `rounds = 1`. The trace confirms it directly through the fifteen sequential `merge:d1`…`merge:d15` labels on rank 0. So this cell shares its depth with the chain and differs only in where the applications happen, which makes the pair the cleanest depth-controlled comparison in the series — and they score 0.875 against 0.6875, differing by five ranks, on identical depth. Both model checks pass: fifteen messages against a predicted fifteen, one round against a predicted one.

The retention difference is not evidence about reduction, and this is the paragraph that should stop anyone quoting it as such. `_surrogate_merge` builds its candidate list from `prompt.splitlines()`, but each input is interpolated as `json.dumps(...)` and contains no newlines, so a binary prompt offers exactly three candidate "items": the prompt's own instruction line containing the example identifier `[F-3-2]`, and the two inputs entire. The unit of loss is an input, not a fact. It then seeds its generator with `seed + len(prompt)`, which makes the draw a deterministic function of prompt length. In the binomial sibling that is catastrophic — its eight depth-1 merges fall into two length classes, five prompts of 1,681 characters and three of 1,701, and replaying the arithmetic shows the five short ones all drop their right-hand input, erasing ranks 1, 3, 5, 7 and 9 together. Here no two prompts have the same length, because the left input is an accumulator that grows at every step, so the fifteen draws are effectively independent and only two came up short: ranks 1 and 10 lost their reports, $64 - 8 = 56$, retention 0.875. The ordering `flat` > `binomial` ≈ `chain` ≈ `kary` in this campaign is therefore an artefact of how many applications drew a repeated prompt length, and the deepest algorithm winning is noise rather than a counter-example.

The second consequence of copying inputs verbatim is the token explosion, and the flat fold maximises it because every one of its applications re-ingests the whole accumulated result. The doubling is almost exact — each output is between 1.94 and 2.09 times its predecessor — and it terminates at a single 1,156,234-token result, more than nine times the 128,000-token context budget the per-rank table records for rank 0. That the run completed anyway is itself a finding: `context_high_water` is 0 and `context_rejections` is 0, because every `msg.recv` in a collective passes `admit=False`, so nothing the reduce moved or produced was ever charged against a context account. A real executor

would have refused this prompt at depth 11 or so. The accounting cannot see it.

Both of the run's remaining substantive numbers are advisory-regime artefacts. At the provenance commit `c5e8e9e` the harness passed the module-level `MERGE_CONTRACT`, which carries no `max_tokens`, so the operator's stated 900-token budget was a sentence in the prompt and nothing checked it; that is why a 1.16-million-token output records `attempt: 1` and the run reports zero contract violations. All four campaigns in my set except `inc-mb-fidelity-inc` are in that regime, and it is the reason none of them can address the question the experiment was built for. The measurement that does address it is `inc-mb-fidelity-inc`'s flat cell: same algorithm, $p = 8$, `max_tokens: 450` enforced with retries, incompressible payloads, and there the fold saturates after three ranks, returns the byte-identical accumulator for four successive applications, and erases ranks 4 through 7 whole at retention 0.3854 — identical to what the depth-2 k-ary tree achieves on the same data. Depth is not the variable; the enforced budget is.

8 Threats to this reading

The executor is a surrogate. Everything above about token growth, cost and retention is about a Python function that concatenates JSON, and none of it transfers to agents. The $38.36 in particular is a modelled price for tokens no model produced, and it should not appear in any cost comparison across executors.

The claim that prompt-length seeding explains the retention ordering is a replay of the kernel's arithmetic, not a logged fact, and it is much better supported for the binomial sibling (where five identical prompt lengths predict the exact observed per-rank vector) than for this run, where I verified only that the fifteen prompt lengths are distinct because the accumulator grows monotonically. The specific reason ranks 1 and 10 rather than any other two were lost I did not reconstruct. What is directly observed here, from the fifteen message payloads under `blobs/`, is that every incoming message is a leaf report carrying exactly one rank's four identifiers — so unlike the chain, no single application in this cell could ever have erased more than one rank, which is a structural reason to expect `flat` to do relatively well under a blob-granular loss model and is worth separating from the seeding argument.

Finally, the depth-15 claim is a claim about the current implementation. It follows from reading `algorithms.py` and is confirmed by fifteen distinctly labelled applications in the trace, but it is a design choice rather than a necessity: the operator carries a `variadic_prompt`, and a `flat` branch that used it would genuinely be a one-shot p-way fold at depth 1. Only `kary` calls the variadic kernel today, which is why the k-ary cells are the only place in the repository where a shallow wide fold is actually measured.